FaroNet

Proyecto 1 - Redes de Computadoras

Ingenieria en Computacion | TEC | II semestre 2026

Jimmy Castillo Guzman
Juan Carlos Gomez

Profesor: Kevin Moraga
Corte: 1 de noviembre de 2026

Informe y anexos

- Diseno, funciones y comandos reproducibles
- Esquema, materiales y presupuesto en colones
- Bitacoras, resultados y autoevaluacion
- Trazabilidad de requisitos y evidencias disponibles

Worker implementado; no utilizado en la demostracion integrada.

Las mediciones pendientes se identifican sin sustituirlas por valores esperados.

Repositorio: https://github.com/JuankGO/redes-proyecto1-faronet

FaroNet: documentación del proyecto

Curso: Redes de Computadoras, Ingeniería en Computación, TEC. **Profesor:** Kevin Moraga. **Periodo:** II semestre de 2026. **Integrantes:** Jimmy Castillo Guzmán y Juan Carlos Gómez. **Repositorio:** https://github.com/JuankGO/redes-proyecto1-faronet **Fecha de corte:** 1 de noviembre de 2026.

Alcance y evidencia

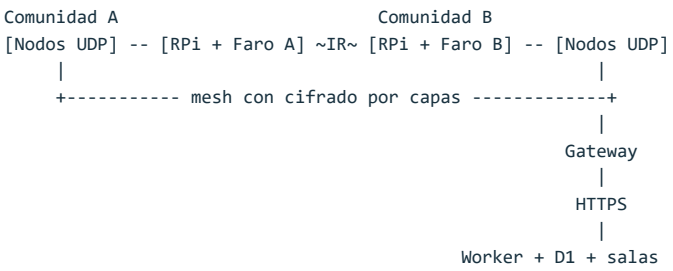
Este documento describe la arquitectura y el uso previsto de FaroNet. Los ejemplos de operación muestran el resultado esperado cuando las partes están operativas. Los resultados medidos se distinguen de esos ejemplos. No se atribuyen voltajes, distancias ni pruebas aprobadas que no se hayan registrado.

Se sigue el enunciado «Proyecto 1 - FaroNet», septiembre de 2026, páginas 3 a 8. La entrega digital incluye fuentes Markdown, RFC ASCII y ambos documentos en PDF. No se requiere imprimirlos. El informe desarrolla los nueve apartados solicitados y los anexos incorporan las tablas y procedimientos completos.

1. Introducción

FaroNet reúne un enlace físico construido por el grupo con una red de mensajes que busca reducir la exposición de contenido y metadatos. Además de enviar texto, hay que reconocer tramas, detectar errores, encontrar nodos, encaminar paquetes y definir qué información puede publicarse en Internet.

La solución se divide en un Faro infrarrojo, una mesh con cifrado por capas y una aplicación de chat. Un gateway puede publicar únicamente mensajes públicos en un Worker. Los mensajes privados permanecen dentro de FaroNet.



Cada Raspberry ejecuta el nodo y controla su placa por GPIO. No se necesita flashear un microcontrolador aparte. Ethernet o WiFi transportan UDP dentro de cada comunidad. Para acreditar el puente exclusivo por luz, las comunidades no deben tener una ruta IP alternativa que evite el Faro.

2. Ambiente de desarrollo

Área	Herramientas y uso
Hardware	Dos Raspberry Pi 2 Model B, microSD y Raspberry Pi OS de 32 bits
Desarrollo	PC, editor de texto o VS Code, Git y GitHub
Capa física	Python y pigpio/pigpiod en GPIO17 y GPIO27
Criptografía	PyNaCl/libsodium y WebCrypto Ed25519
Red	Sockets UDP y HTTPS
Edge	TypeScript, Node.js, Wrangler, D1 y Durable Objects
Pruebas	pytest, Vitest, TypeScript y multímetro

Área	Herramientas y uso
Montaje	MB102, placa perforada, cautín, estaño, pinzas y alambre
Documentación	Markdown, RFC ASCII y ReportLab para PDF

El control de versiones conserva cambios por autor, fecha y propósito. Cada cambio se revisa con `git diff` y las pruebas correspondientes. Los secretos y registros de usuarios no se incorporan al repositorio. El número de commits no se usa para calcular horas.

La depuración se separa en lectura estática del detector, recepción de bytes, tramas válidas con CRC y tráfico mesh. Una lectura 0/1 o un LED encendido no demuestran una trama recibida. Las evidencias de laboratorio no se convierten en registros persistentes de tráfico del sistema.

3. Diseño

3.1 Circuito y conexión al nodo

Cada Faro tiene un LED IR con resistencia de 330 ohmios, un NPN 2N2222A, un fototransistor PT333, dos indicadores y un botón. GPIO17 controla la base del NPN mediante 2,2 kiloohmios. Otra resistencia de 10 kiloohmios une base y GND. El receptor conecta el colector a RX y a 3,3 V mediante 10 kiloohmios; su emisor va a GND. Un condensador de 100 nF desacopla alimentación y tierra.

RX vale 1 sin luz y 0 con luz. `SeriePigpio` invierte la lectura y produce UART 8N1 por software. Se conservan GPIO17/27 del prototipo. La Raspberry mantiene su fuente micro-USB normal y la placa toma 3,3 V de su pin 1. La propuesta de usar 5 V en el emisor no se considera una modificación implementada.

Los headers permiten retirar la Raspberry sin desoldar. El montaje final debe estar soldado: las fotos de protoboard documentan desarrollo, no sustituyen las cuatro vistas de los Faros terminados que pide el enunciado.

3.2 Protocolo físico

La trama contiene 12 bytes de cabecera, hasta 114 de contenido y 2 de CRC, con máximo de 128 bytes. Declara versión, tipo, destino, origen, secuencia y longitud. CRC-16/CCITT-FALSE detecta errores accidentales; no cifra ni autentica frente a un atacante.

Manchester se aplica al preámbulo, delimitador y trama. Una trama máxima produce 266 bytes UART, más un byte auxiliar de pigpio, sin modificar el límite de 128 bytes de la trama lógica. UART agrega los bits de inicio y parada.

El adaptador espera canal libre y usa retroceso aleatorio. La entrega dirigida usa ACK y reintentos; la difusión no se confirma. Los duplicados se descartan. La ocupación se estima con bytes recibidos y silencio, no con un detector analógico de portadora; pueden existir colisiones.

3.3 Mesh, enrutamiento y privacidad

Los nodos anuncian vecinos, intercambian entradas firmadas del directorio y propagan rutas por costo en saltos. Cada entrada identifica versión, clave, dirección de transporte y vencimiento. Una firma verifica quién publicó la entrada, no que cada persona controle una sola identidad.

Se usan celdas de 1024 bytes con relleno y cebolla de tres relays mediante SealedBox. Cada relay abre una capa y obtiene el próximo salto. El código exige cinco candidatos distintos del origen y el destino, así que la demostración anónima requiere siete nodos. Pueden ser procesos separados en una PC, cada uno con su puerto y carpeta de llaves.

El enlace Faro divide los mensajes en fragmentos de hasta 110 bytes más cuatro bytes de índice y total. Una celda binaria de 1024 bytes ocupa diez fragmentos. El sobre JSON que codifica la celda en hexadecimal puede necesitar más; no debe confundirse su tamaño con el de la celda original.

3.4 Aplicación y edge

El mensaje de aplicación mide 256 bytes e incluye versión, visibilidad, tema, hora, longitud, texto y relleno. El chat distingue privados, temas privados y mensajes públicos. Argon2id deriva las claves de temas a partir de frases.

El gateway filtra mensajes públicos y firma peticiones HTTPS. El Worker verifica Ed25519, hora y nonce; implementa salas mediante Durable Objects y WebSockets, feeds Atom, directorio verificable y un buzón de blobs de 1024 bytes con borrado al leer y TTL. Su código y configuración están presentes.

URL informada por el grupo: https://faronet-worker.faronet-jimmy-juan.workers.dev

El Worker no se utilizó en la demostración integrada. Tener código, una URL y pruebas locales no acredita el recorrido mesh-gateway-edge en producción.

3.5 Modelo de amenazas

Adversario	Observa	Defensa y límite
Relay malicioso	Vecinos, próximo salto y tiempos	Cebolla y padding; la colusión permite correlación
Observador óptico	Luz, duración y cabeceras	Cifrado superior y dirección rotable; la demo cruda no es privada
Operador del gateway	Públicos y tráfico al edge	Los privados no se exportan; conserva visibilidad de su conexión
Proveedor del edge	IP de conexión, tiempos y públicos	La aplicación no registra encabezados; el proveedor sí ve su infraestructura
Acceso a microSD o sistema	Llaves y memoria	Pánico y borrado lógico; no garantizan borrado forense en flash

El directorio conserva direcciones operativas para enrutar. No es una bitácora histórica, pero expone metadatos. No hay protección completa contra observación global, Sybil, denegación de servicio o un sistema operativo comprometido. HELLO y RUTA no están firmados en v1. La pista del emisor de un privado tampoco autentica su identidad.

4. Estructuras de datos y funciones

Módulo	Estructura o función	Uso
`faro.frame`	`Trama`, `codificar`, `decodificar`, `crc16`	Formato y validación
`faro.linea`	`armar_senal`, `extraer_trama`	Manchester y sincronización
`faro.serie_pigpio`	`SeriePigpio`	UART GPIO17/27 invertida
`faro.dispositivo_luz_adaptador`	`DispositivoLuzAdaptador`	Canal, ACK y tramas externas
`faro.identidad`, `direccion`, `panico`	Llaves y dirección	Crear, rotar y borrar
`mesh.vecinos`, `rutas`, `directorio`	Tablas en memoria	Descubrir, encaminar y verificar
`mesh.onion`, `celda`, `saltos`	Cebolla y celda fija	Cifrado y selección de relays
`mesh.enlace_faro`	Cola y parciales	Fragmentación y reensamblado
`app.mensaje`, `cliente`, `comandos`	Mensajes y órdenes	Chat y público/privado
`gateway.gateway`, `firma`	Cola y firma	Exportar solo públicos

Módulo	Estructura o función	Uso
`worker/src/index.ts`	`Sala`, HTTP y cron	Salas, feeds, buzón y limpieza

El anexo de funciones describe entradas, salidas, errores y límites. El código conserva docstrings y comentarios; los módulos antiguos de prueba cruda se retiran para no mantener un segundo mapa de pines.

5. Instrucciones de ejecución

El manual anexo contiene los comandos de instalación, pruebas ópticas, mensajería, mesh y despliegue. El Faro ejecuta Python sobre Linux; no requiere compilación ni flasheo independiente. Debe existir un único proceso dueño de GPIO27 en cada Raspberry.

```
sudo systemctl enable --now pigpiod
python3 -m pytest tests -q
python3 -m faro.medir loopback --uart pigpio --baudios 300 --tramas 5
```

Las demos de texto del Faro diagnostican el enlace y no reemplazan el chat cifrado. La demostración completa debe verificar primero hardware y después mesh, privacidad y publicación.

6. Actividades realizadas por estudiante

El grupo estimó unas 20 horas-persona entre ambos durante tres fines de semana. La bitácora reconstruida distribuye diez horas a cada estudiante en materiales, montaje, soldadura, código, pruebas e integración. Fechas, reparto y duración por actividad son aproximaciones para revisión, no un cronómetro histórico.

Se interpretaron los tres fines de semana como 17-18 y 24-25 de octubre y 31 de octubre-1 de noviembre de 2026. Si el grupo se refería a otros fines de semana, debe corregir las fechas. El registro anterior de 1 h sin autor se conserva aparte y no se suma a las 20 h para evitar duplicar tiempo.

El anexo contiene una línea por actividad, fecha y horas y suma por integrante. El reporte Git presenta contribuciones reales; no se utiliza para inventar horarios ni asistencia.

7. Autoevaluación y estado final

El proyecto tiene implementaciones de las capas solicitadas y pruebas de software. La bitácora identifica cada ejecución y sus resultados. El grupo reportó un loopback de protoboard estable a 1000 baudios, pero eso no equivale a medir tasa útil y BER a tres distancias.

En el diagnóstico por SSH del 1 de noviembre no se confirmó el enlace entre Faros. Se generaron pulsos de salida, pero no se recibió una secuencia útil. Las entradas medidas requieren revisar acondicionamiento y alineación; la causa única no está determinada. No se declara como aprobada una comunicación que solo se describió de forma hipotética.

La autoevaluación anexa cubre la rúbrica 0 a 10 y la escala de aprendizaje 1 a 5. Sus notas son una propuesta editable que deben validar ambos integrantes. El Worker figura implementado y probado localmente cuando existe esa evidencia, pero no utilizado en la integración final.

8. Lecciones aprendidas

- Separar lectura estática, bytes, CRC y tráfico de aplicación al depurar.
- Mantener los mismos GPIO y la misma polaridad en código, esquema y manual.
- Confirmar la recepción en el otro equipo; «enviado» no garantiza entrega.
- Reservar margen respecto de la mayor velocidad que pasa una prueba cercana.
- Registrar continuidad y voltajes antes de conectar una placa soldada.

- Aislar el detector del emisor propio y reducir reflejos e iluminación externa.
- Probar por separado criptografía, transporte, aplicación y despliegue.
- Anotar horas y mediciones durante el trabajo, no al final.
- Describir límites de anonimato y borrado en microSD sin prometer seguridad absoluta.
- Preparar la defensa de veinte minutos con comandos y evidencias disponibles.

9. Bibliografía

- Moraga, K. «Proyecto 1: FaroNet», TEC, septiembre de 2026, pp. 3-9. Enunciado del curso.
- RFC Editor. RFC 7322, *RFC Style Guide*. https://www.rfc-editor.org/rfc/rfc7322
- RFC Editor. RFC 6973, *Privacy Considerations for Internet Protocols*. https://www.rfc-editor.org/rfc/rfc6973
- RFC Editor. RFC 8032, *Edwards-Curve Digital Signature Algorithm*. https://www.rfc-editor.org/rfc/rfc8032
- PyNaCl. https://pynacl.readthedocs.io/en/latest/
- pigpio, interfaz Python. https://abyz.me.uk/rpi/pigpio/python.html
- Raspberry Pi, documentación GPIO. https://www.raspberrypi.com/documentation/computers/raspberry-pi.html
- Cloudflare, Wrangler. https://developers.cloudflare.com/workers/wrangler/commands/
- Cloudflare, D1 y migraciones. https://developers.cloudflare.com/d1/wrangler-commands/
- Cloudflare, Durable Objects. https://developers.cloudflare.com/durable-objects/best-practices/rules-of-durable-objects/
- Historial y código del grupo. https://github.com/JuankGO/redes-proyecto1-faronet
- Presupuesto histórico del grupo, 13 de octubre de 2026, conservado en `docs/hardware/FaroNet_Materiales.pdf`. La BOM actual distingue los valores del montaje nuevo.

Manual de ejecución y demostración

Los comandos se ejecutan desde la raíz del repositorio, salvo donde se indica `worker`. Usar terminales diferentes para procesos que deben permanecer activos. Las direcciones IP de los ejemplos son de laboratorio; reemplazarlas por las de la red propia. Ningún resultado esperado de esta guía se considera una medición.

1. Preparar cada Raspberry

Grabar Raspberry Pi OS Lite de 32 bits en la microSD con Raspberry Pi Imager, configurar usuario y SSH y conectar Ethernet. La Pi 2 no incorpora WiFi; si se usa WiFi hace falta un adaptador. No se flashea un firmware adicional: Python controla los GPIO desde Linux.

```
sudo apt update
sudo apt install -y git pigpiod python3-pigpio python3-nacl python3-serial python3-gpiozero python3-pytest
sudo systemctl enable --now pigpiod
git clone https://github.com/JuankGO/redes-proyecto1-faronet.git faronet-juank
cd faronet-juank
pigs t
python3 -m pytest tests -q
```

`pigs t` devuelve un contador si pigpiod responde. El montaje usa TX GPIO17 y RX GPIO27, no `/dev/serial0` ni `/dev/ttyAMA0`. No habilitar la consola serie para estas pruebas. Conectar o modificar las placas solo con la alimentación retirada.

Si ya hay una copia parcheada con cambios locales, conservarla. Clonar esta entrega en otra carpeta y usar esa carpeta en los comandos, en vez de forzar un `git pull` o descartar modificaciones.

2. PC, SSH y pruebas sin hardware

```
python -m pip install -r requirements.txt
python -m pytest tests -q
python -m app.demo_chat
python -m mesh.demo_nodo
python -m mesh.demo_onion
python demo_mesh_chat.py
```

Las demos y el Faro simulado usan sockets locales. No prueban luz ni soldaduras. Para controlar las Raspberry desde dos ventanas del PC:

```
ssh faro-a@faro-a.local
ssh faro-b@faro-b.local
```

Cada línea va en una ventana distinta. Si mDNS no resuelve, consultar `hostname -I` en cada Raspberry y sustituir el nombre por su IP. SSH transporta los comandos; un mensaje enviado mediante `--uart pigpio` debe cruzar por luz.

3. Verificar un Faro

Completar primero continuidad y ausencia de cortos de la bitácora de multímetro. Para la prueba estática, no dejar otro programa usando los GPIO:

```
pigs modes 27 r
pigs modes 17 w
pigs w 17 0
pigs r 27
pigs w 17 1
pigs r 27
pigs w 17 0
```

Con el emisor y detector de esa placa enfrentados se espera 1 con IR apagado y 0 con IR encendido. Si la entrada solo cambia al activar `pud`, revisar la conexión RX, el pull-up externo y los voltajes. Una entrada flotante no demuestra recepción.

Prueba de tramas en la misma placa:

```
python3 -m faro.medir loopback --uart pigpio --baudios 300 --tramas 5
```

Esperado: cinco tramas `OK`, cero bits distintos y BER cero. No forzar las patas soldadas para enfrentarlas; planear la orientación o probar con el otro Faro. Para detener un programa: Ctrl+C. Si queda un lector abierto después de un error, detener primero todos los programas del Faro y ejecutar `pigs slrc 27`; si indica que no estaba abierto, no hay lector que liberar.

4. Medir dos Faros

Alinear IR A con PT333 B e IR B con PT333 A. Evitar luz del emisor propio en cada detector. Ambas Raspberry usan los mismos GPIO y baudios. No deben compartir alimentación de placas ni estar unidas por señales TX/RX eléctricas.

Primero en B:

```
python3 -u -m faro.medir recibir --uart pigpio --baudios 300 --tramas 100 --espera 120 --distancia 0.01
```

Después en A:

```
python3 -u -m faro.medir enviar --uart pigpio --baudios 300 --tramas 100 --pausa 1
```

La pausa de un segundo supera los 20 tiempos de byte que usa el receptor para separar ráfagas a 300 baudios. Sin esa pausa puede interpretar varias tramas como una sola señal. No enviar más de 256 tramas por ejecución de BER: SEQ tiene un byte y el receptor cuenta una sola vez cada secuencia.

Repetir a 0,02 m y 0,05 m y luego intercambiar A y B. Son tres distancias de referencia para el procedimiento, no tres resultados ya obtenidos. A 300 baudios 100 tramas máximas y pausas tardan aproximadamente 17 minutos. Para ensayar antes, usar diez tramas en ambos extremos. Mantener luz ambiental normal y registrar hora, iluminación, separación, alineación y resistencia del receptor.

Para tasa útil con confirmación, primero en B:

```
python3 -u -m faro.medir eco --uart pigpio --baudios 300
```

Después en A:

```
python3 -u -m faro.medir tasa --uart pigpio --baudios 300 --tramas 20 --espera-ack 10 --distancia 0.01
```

Detener el eco al terminar. Repetir a las otras dos distancias y en el sentido contrario. Registrar bytes confirmados, tiempo y reintentos, no solo baudios. Un metro se requiere para competir por el extra de velocidad, no para las tres distancias obligatorias. Si se busca el extra, agregar esa medición real.

5. Enviar texto de prueba

Primero en B:

```
python3 -u -m faro.demo_recibir --uart pigpio --baudios 300
```

En A:

```
python3 -u -m faro.demo_enviar --uart pigpio --baudios 300 --destino ffffffff --mensaje "Hola B, soy el Faro A"
```

«Enviado a difusión» solo confirma que A transmitió. B debe mostrar `Recibido de` y el texto para acreditar entrega. Para ACK, sustituir `ffffffff` por la dirección de ocho dígitos hexadecimales que imprime B. No usar la IP como destino físico.

Para responder, detener con Ctrl+C el receptor B, iniciar `demo_recibir` en A y ejecutar `demo_enviar` en B. No ejecutar ambos programas a la vez en una misma Pi. Estas demos envían texto de diagnóstico sin el cifrado de aplicación.

6. Chat, mesh y siete nodos

Prueba funcional corta

Para probar el chat sin hardware, ejecutar sus demos. `--permitir-directo` permite ensayar con pocos nodos, pero desactiva la exigencia de relays: no acredita el anonimato solicitado y debe omitirse en la demostración final.

Topología de referencia con enlace óptico exclusivo

Se proponen dos chats que poseen los Faros, cuatro relays UDP y un gateway: siete procesos. Los relays y el gateway pueden correr en una PC de la comunidad A. La comunidad B no debe tener conectividad IP de aplicación hacia A; su único camino para la mesh es el Faro. SSH de administración no debe convertirse en semilla, túnel o ruta de datos de la mesh.

En cada terminal definir las IP de laboratorio que correspondan:

```
IP_A=192.168.10.10
IP_B=192.168.20.10
IP_PC=192.168.10.20
```

En la Raspberry A:

```
python3 -m app.chat --nodo "$IP_A:7001" --alias faro-a --uart pigpio --baudios 300 --semilla "$IP_PC:7101"
  --carpeta ~/.faronet-a
```

En la Raspberry B, sin una semilla IP hacia la comunidad A:

```
python3 -m app.chat --nodo "$IP_B:7002" --alias faro-b --uart pigpio --baudios 300 --carpeta ~/.faronet-b
```

En cuatro terminales de la PC, definir IP_A e IP_PC y ejecutar una línea por terminal:

```
python3 -m mesh.nodo --puerto 7101 --ip-publica "$IP_PC" --semilla "$IP_A:7001" --carpeta ~/.faronet-r1
python3 -m mesh.nodo --puerto 7102 --ip-publica "$IP_PC" --semilla "$IP_A:7001" --carpeta ~/.faronet-r2
python3 -m mesh.nodo --puerto 7103 --ip-publica "$IP_PC" --semilla "$IP_A:7001" --carpeta ~/.faronet-r3
python3 -m mesh.nodo --puerto 7104 --ip-publica "$IP_PC" --semilla "$IP_A:7001" --carpeta ~/.faronet-r4
```

El séptimo proceso es el gateway de la sección siguiente. Cada proceso tiene su propia carpeta, incluso si comparte máquina. Antes de enviar, esperar a que el directorio conozca cinco candidatos además de los dos extremos. El enlace lento puede retrasar los anuncios y el reensamblado; esta topología es una guía de reproducción, no una integración óptica ya validada.

Dentro de cada chat:

```
/nodos
/priv PREFIJO_DEL_ID_DESTINO Hola desde FaroNet
/unirse equipo frase-compartida-larga
/tema equipo Mensaje del tema privado
/publico noticias Mensaje publico de demostracion
/panico
/salir
```

Reemplazar el prefijo por el ID que muestre `/nodos`. Compartir la frase del tema por un canal aparte. `/panico` borra estado y cambia identidad; comprobarlo con datos de prueba, no con llaves que se necesiten conservar. El botón físico de GPIO24 lo ejecuta mientras corre el nodo/chat/gateway con `--uart`.

7. Worker implementado, no utilizado en la integración

URL informada: https://faronet-worker.faronet-jimmy-juan.workers.dev

Los pasos siguientes permiten reproducir el despliegue; no implican que el grupo haya realizado la demostración mesh-gateway-edge. El repositorio contiene código, migraciones y configuración. Se debe usar el plan gratuito y verificar su vigencia antes de desplegar. No se despliega automáticamente al generar documentos.

En una PC con Node.js y npm:

```
cd worker
npm ci
npx tsc --noEmit
npx vitest run
npx wrangler login
```

Para una cuenta nueva, crear D1 y sustituir `database_id` en `wrangler.jsonc` por el que devuelva la cuenta; no reutilizar el de otra persona:

```
npx wrangler d1 create faronet-db
npx wrangler d1 migrations apply faronet-db --remote
```

Crear la identidad persistente del gateway desde la raíz del proyecto:

```
python3 -c "from pathlib import Path; from faro.identidad import cargar_o_crear;
  firma,_=cargar_o_crear(Path.home()/'.faronet-gw'); print(firma.verify_key.encode().hex())"
```

Copiar únicamente esa llave pública al secreto del Worker. Nunca copiar el archivo de llave privada. Desde `worker`:

```
npx wrangler secret put GATEWAY_PUB
npx wrangler deploy
```

Si se configura el secreto antes de la primera publicación, Wrangler puede crear el Worker durante ese paso. Usar la URL real que imprima el despliegue. Para desarrollo local: copiar `.dev.vars.example` a `.dev.vars`, completar la llave pública y ejecutar `npx wrangler dev`. Ese archivo está excluido de Git.

El séptimo proceso, en la misma PC y con la misma carpeta de identidad creada:

```
python3 -m gateway.gateway --puerto 7200 --ip-publica "$IP_PC" --semilla "$IP_A:7001" --carpeta ~/.faronet-gw
  --worker https://faronet-worker.faronet-jimmy-juan.workers.dev
```

Antes de publicar comprobar que GATEWAY_PUB corresponde a esa carpeta. Probar:

```
https://faronet-worker.faronet-jimmy-juan.workers.dev/sala/noticias
https://faronet-worker.faronet-jimmy-juan.workers.dev/feed/noticias.atom
https://faronet-worker.faronet-jimmy-juan.workers.dev/directorio
```

Para el buzón local, desde la raíz:

```
python3 tools/prueba_buzon.py http://localhost:8787
```

Esperado: depósito 201, primera lectura 200 con descifrado, segunda lectura 404 y rechazo 413 para tamaño incorrecto. El secreto compartido se genera en memoria.

8. Defensa de veinte minutos

Minutos	Demostración y evidencia
0-4	Mostrar Faros soldados, ambas caras, headers, esquema y multímetro
4-7	Tramas ópticas y tabla ya medida de BER/tasa a tres distancias
7-11	Directorio, rutas, tres relays y mensaje privado; cortar luz y comprobar dependencia
11-14	Tema privado, padding, rotación y pánico con indicadores
14-17	Mostrar código y pruebas del Worker; aclarar que no se utilizó integrado
17-20	RFC, bitácoras, limitaciones y preguntas

No intentar medir cien tramas a tres distancias dentro de la defensa: llevar resultados previos. Mostrar el estado real del enlace aunque una prueba falle.

Estructuras de datos y funciones principales

Los nombres corresponden al código del repositorio. Enteros binarios de varios bytes usan orden de red, big-endian. Los detalles normativos están en el RFC.

Faro: formato y línea

`Trama` es una dataclass con `tipo`, `dst`, `src`, `seq`, `payload` y `version`.

Campo	Tamaño	Regla
VER	1 byte	Versión 1; receptor rechaza versiones no soportadas
TIPO	1 byte	DATA=1, ACK=2, BEACON=3
DST y SRC	4 bytes cada uno	FFFFFFFF es difusión; cero no es identidad asignable
SEQ	1 byte	Secuencia módulo 256; ACK repite la secuencia
LEN	1 byte	Longitud de PAYLOAD, de 0 a 114
PAYLOAD	LEN bytes	Contenido de la capa superior
CRC	2 bytes	CRC-16/CCITT-FALSE de cabecera y contenido

Función	Entrada y salida	Error o límite
`frame.codificar(t)`	Trama a bytes	Valida tamaños de direcciones y contenido
`frame.decodificar(crudo)`	Bytes a Trama	`TramaError` por versión, largo o CRC
`frame.crc16(datos)`	Bytes a entero de 16 bits	Vector 123456789 produce 29B1
`linea.armar_senal(trama)`	Bytes a lista de medios bits	Añade cuatro AA y SFD D5, luego Manchester
`linea.extraer_trama(medios)`	Medios bits a bytes de trama	`LineaError` si no hay pares válidos, sincronización o longitud

Manchester representa 1 por 01 y 0 por 10. `bits_a_bytes` y `bytes_a_bits` convierten sin cifrar. Una trama externa construida con Scapy u otra herramienta debe respetar el mismo formato y CRC; se entrega a `enviar_crudo`.

Puerto e interfaz DispositivoLuzAdaptador

`Medio` define `enviar`, `recibir`, `ocupado` y `cerrar`. `MedioSimUDP` transporta medios bits por UDP y `MedioUART` usa un objeto compatible con pyserial. `--uart pigpio` crea `SeriePigpio` con TX17, RX27 e inversión de recepción.

`SeriePigpio` mantiene un `bytearray` de entrada. Implementa `write`, `flush`, `read`, `in_waiting`, `reset_input_buffer` y `close`. Sus ondas son generadas por pigpiod; no dependen de un bucle de Python para cada bit. Agrega 55 hexadecimal como relleno de transporte, fuera del tamaño de trama. `MedioUART` delimita ráfagas por un silencio mínimo de veinte tiempos de byte.

Método del adaptador	Comportamiento
`enviar(dst, payload, tipo)`	Emite una trama sin esperar ACK; devuelve SEQ
`enviar_confiable(dst, payload)`	Espera ACK y reintenta; devuelve bool
`enviar_crudo(trama_bytes)`	Valida una trama externa antes de transmitir
`recibir(timeout)`	Devuelve DATA/BEACON o None; confirma DATA dirigida y filtra duplicados
`cambiar_direccion(nueva)`	Cambia la dirección, vacía duplicados y anuncia BEACON

El adaptador guarda secuencia, cola `deque` de pendientes, último SEQ por origen y contadores de enviadas, recibidas, descartadas y reintentos. Estos contadores son agregados, no una bitácora persistente de usuarios. Una sola hebra usa la instancia. Los errores de tramas recibidas se descartan; errores de acceso al puerto deben diagnosticarse antes de ejecutar la mesh.

Identidad, indicadores y pánico

`identidad.cargar_o_crear(carpeta)` devuelve una SigningKey Ed25519 y una PrivateKey de cifrado, mediante PyNaCl. Los archivos se escriben con permisos privados en sistemas que los soportan. `direccion.cargar_o_crear` conserva una dirección de cuatro bytes; `rotar` genera otra evitando cero y difusión.

`Indicadores` controla TX22 y RX23 y permite parpadeo de pánico. `BotonPanico` usa GPIO24 con pull-up, antirrebote y bloqueo para evitar llamadas simultáneas. `panico.panico` sobrescribe y elimina archivos sensibles conocidos. `Nodo.panico` también vacía tablas y colas, renueva identidad y avisa al cliente. La sobrescritura lógica no garantiza eliminación física en microSD ni de todas las copias que el intérprete pudo mantener en memoria.

Vecinos, rutas y directorio

`TablaVecinos` asocia ID a canal, dirección y última observación. Registra HELLO de vecinos directos y vence entradas sin actividad. No convierte un anuncio recibido por un tercero en una vecindad física directa.

`TablaRutas` guarda destino, siguiente salto, costo y vigencia. Aprende vecinos, mezcla anuncios, evita valores mal formados y olvida rutas vencidas. El infinito es 16. Es vector de distancias, no un protocolo de consenso ni una garantía de camino óptimo frente a nodos hostiles.

`Directorio` reúne entradas con versión, ID Ed25519, clave pública de cifrado, canal, dirección, creación, TTL y rol opcional. `crear_entrada`, `firmar`, `verificar` y `validar` comprueban estructura, firma y vigencia. `a_bytes` produce JSON canónico con claves ordenadas y representación ASCII compatible con el Worker. Se expiran entradas y se consulta a gateways por su rol.

Celdas, cebolla y fragmentación

`empaquetar_celda` antepone longitud de dos bytes y rellena aleatoriamente hasta 1024. `desempaquetar_celda` valida y recupera contenido. Las primitivas de cifrado son de PyNaCl. `construir_cebolla` cifra para destino y relays; `abrir_como_relay` obtiene el siguiente salto, mientras `abrir_como_destino` recupera el mensaje.

`elegir_saltos` usa SystemRandom, excluye origen, destino y duplicados y exige cinco candidatos para elegir tres relays. Si no los hay lanza `SaltosError`. `--permitir-directo` es una excepción de pruebas y no ofrece esa propiedad.

`EnlaceFaro` tiene una cola de hasta 256 mensajes y una tabla parcial por origen. Cada fragmento lleva índice y total de dos bytes cada uno, más hasta 110 de datos. `fragmentar`, `_recibir_fragmento` y `_purgar` parten, reúnen y vencen mensajes. El máximo de mensaje es 64 KiB y el timeout parcial es diez segundos sin actualizar. No incluye un identificador de mensaje independiente: no se deben intercalar mensajes fragmentados del mismo origen.

`Nodo` coordina socket, tablas, enlace, mantenimiento y recepción con bloqueos y colas. HELLO y rutas se anuncian periódicamente; dirección y entradas se rotan o renuevan. Su configuración se recibe de argparse. Los mensajes recibidos viven una hora y se mantiene un límite de memoria; no se escriben conversaciones a disco.

Aplicación

`empaquetar_mensaje` crea exactamente 256 bytes: VER1, VIS1, TEMA8, HORA4, LARGO2, texto y relleno. El máximo de texto es 240 bytes antes del envoltorio de un tema público. `desempaquetar_mensaje` comprueba tamaño, visibilidad, longitud, TTL de una hora y tolerancia de reloj. `hash_tema` usa BLAKE2b de ocho bytes.

`Cliente` mantiene alias local, temas y claves en memoria. Ofrece `enviar_privado`, `enviar_tema`, `enviar_publico`, `recibir_pendientes` y `olvidar_todo`. `clave_de_tema` deriva una clave con Argon2id de nombre y frase. El alias y la pista de emisor no constituyen una firma de identidad. `parsear_comando` valida la sintaxis de `/priv`, `/tema`, `/publico` y pánico.

Gateway y Worker

`Gateway` consume mensajes, valida visibilidad y tema y encola públicos. Usa reintentos para errores temporales del edge; privados y contenido inválido no se publican. `encabezados_firmados` firma la cadena FARONET-GW-1, método, ruta, timestamp, nonce y SHA-256 del cuerpo. No se implementan primitivas propias.

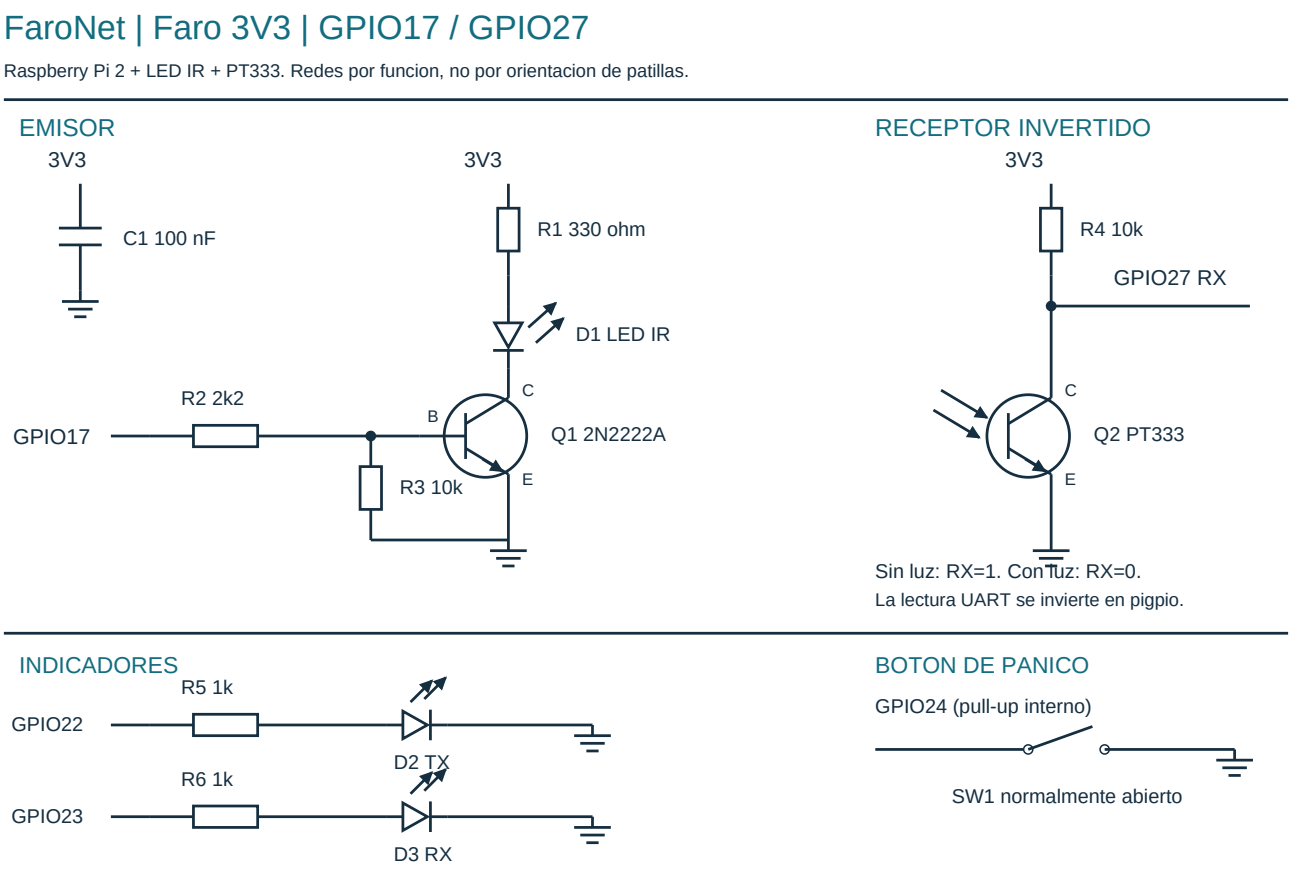
Endpoint	Función y control
POST `/api/relay`	Publicar un mensaje público; firma del gateway
POST `/api/directorio`	Recibir entradas y verificar firmas
GET `/directorio`	Réplica de lectura con firmas y expiración
GET `/sala/tema`	Página del tema
GET `/sala/tema/ws`	WebSocket de lectura en Durable Object Sala
GET `/feed/tema.atom`	Feed Atom con XML escapado
PUT/GET `/buzon/id`	Blob fijo, ID opaco, TTL y eliminación al leer

D1 conserva `mensajes`, `buzon`, `directorio` y `nonces`, con índices y campos de expiración. La ventana de firma es de 60 s, nonce 120 s, blob 24 h y públicos siete días. El cron elimina vencidos. El límite configurado es 60 peticiones por minuto por clave de cliente; es un parámetro del proyecto, no una cuota del plan. La aplicación no persiste CF-Connecting-IP ni User-Agent, pero el directorio incluye direcciones de transporte dentro de las entradas publicadas.

Las pruebas verifican límites y rechazos sin probar anonimato absoluto. La operación integrada del Worker no se utilizó en la demostración del grupo.

Esquemático y conexiones del Faro actual

Versión de referencia: Raspberry Pi 2, 3,3 V, TX17/RX27 y PT333 con pull-up. La numeración R1-R6 de este documento corresponde a esta revisión.



Esquemático del Faro a 3,3 V

Redes eléctricas

```
EMISOR
3V3 -- R1 330 ohm -- anodo D1 LED IR catodo -- C de Q1 2N2222A
GPIO17 -- R2 2k2 --+-- B de Q1
                   |
                  R3 10k
                   |
GND ---------------+-- E de Q1

RECEPTOR
3V3 -- R4 10k --+-- GPIO27 (RX)
                |
                C de Q2 PT333
                E
                |
GND ------------+

INDICADORES
GPIO22 -- R5 1k -- anodo D2 TX catodo -- GND
GPIO23 -- R6 1k -- anodo D3 RX catodo -- GND

PANICO Y DESACOPLO
GPIO24 -- SW1 normalmente abierto -- GND
3V3 --- C1 100 nF --- GND
```

R1 también puede estar entre el cátodo de D1 y el colector de Q1 si permanece en la misma rama serie y no hay otra conexión en el punto intermedio. Es la disposición equivalente usada en la guía de placa perforada. No unir el LED directamente a la alimentación sin resistencia.

Header desmontable

Contacto J1	Red	Pin físico Raspberry	BCM
1	3V3	1	-
2	GND	6	-
3	TX	11	17
4	RX	13	27
5	LED_TX	15	22
6	LED_RX	16	23
7	PANICO	18	24

No confundir número BCM con posición física. No conectar el header de la placa a los siete pines consecutivos de la Raspberry: cada cable va al pin indicado. El botón usa pull-up interno y debe unir GPIO24 con GND al presionarse. En un pulsador de cuatro patas, comprobar con continuidad cuáles ya están unidas.

Polaridad y orientación

Q1 se conecta por función B/C/E, no por una regla universal de lado plano. El patillaje depende del fabricante y encapsulado. Identificarlo con la referencia de la pieza o prueba de diodo antes de soldar. En Q2 también se confirma C/E; no generalizar longitud de pata o color a todos los fototransistores.

En oscuridad el colector de Q2 debe quedar cerca de 3,3 V. Con luz suficiente debe bajar cerca de GND. Ese nivel se invierte dentro del lector pigpio. El condensador C1 y el pull-up R4 no deben quedar sin conexión al riel de 3,3 V.

Soldadura, revisión y fotografías

Cada unión de la protoboard se reproduce mediante alambre o pista; los agujeros de una placa de islas no comparten conexión por filas. Usar headers soldados, evitar puentes accidentales y dejar acceso óptico al borde. El detector se aísla del LED propio mediante separación y material opaco.

La foto del Faro B muestra la cara de componentes de una placa soldada. Faltan evidencias completas de ambas caras de A y B. No se certifica continuidad ni calidad del cobre mediante una imagen de componentes. Registrar la comprobación en la bitácora de multímetro.

Lista de materiales y presupuesto

Montaje actual: dos Faros para Raspberry Pi 2, GPIO17/27, PT333 y electrónica a 3,3 V. Cada Faro requiere **seis resistencias** y un condensador, además de los semiconductores, indicadores, botón y conectores. Esta lista sustituye como guía de montaje al PDF histórico `FaroNet_Materiales.pdf`, que corresponde a otros valores de resistencias y a un receptor de polaridad distinta.

Componentes por placa y para el par

Los precios son **estimaciones de presupuesto en colones**, no una factura ni una cotización vigente. Permiten valorar piezas que vienen de un kit sin precio unitario conocido. No incluyen transporte, impuestos adicionales ni Raspberry. Se deben sustituir por el costo real si se dispone del comprobante.

Ref.	Material	Por Faro	Para dos	Precio unitario CRC	Subtotal dos CRC
D1	LED IR compatible con PT333, nominalmente 940 nm	1	2	350	700
Q2	Fototransistor PT333 negro	1	2	900	1800
Q1	NPN 2N2222A	1	2	447	894
R1	Resistencia 330 ohmios, 1/4 W	1	2	50	100
R2	Resistencia 2,2 kiloohmios, 1/4 W	1	2	50	100
R3, R4	Resistencia 10 kiloohmios, 1/4 W	2	4	50	200
R5, R6	Resistencia 1 kiloohmio, 1/4 W	2	4	50	200
C1	Cerámico 100 nF, código 104	1	2	100	200
D2, D3	LED visible TX y RX, colores disponibles	2	4	100	400
SW1	Pulsador normalmente abierto	1	2	250	500
J1	Tira de header de siete contactos, paso 2,54 mm	1	2	200	400
Cableado	Cables desmontables compatibles con los headers	7	14	50	700
Placa	Perforada de islas, HS-03 o equivalente	1	2	1000	2000

Ref.	Material	Por Faro	Para dos	Precio unitario CRC	Subtotal dos CRC
Puentes	Porción de alambre aislado por placa	1	2	200	400
Consumible	Porción de estaño y flux por placa	1	2	250	500
TOTAL	Valor estimado de ambos circuitos	-	-	-	**9094**

Valor estimado por Faro: **CRC 4547**. El precio de Q1 toma como referencia el presupuesto anterior (USD 0,95 a CRC 470/USD, redondeado); los demás son asignaciones orientativas. No se afirma que el PT333 ni la placa HS-03 se hayan comprado a esos precios. El emisor debe identificarse por su referencia real; un encapsulado transparente no demuestra por sí solo su longitud de onda.

Equipos reutilizados y herramientas

Recurso	Cantidad	Tratamiento del costo
Raspberry Pi 2 Model B, 1 GB	2	Ya disponibles; valor de reposición no cotizado
MicroSD con Raspberry Pi OS	2	Ya disponibles; no se necesita una capacidad específica de 256 GB
Fuente micro-USB apropiada para cada Raspberry	2	Reutilizada si está disponible
MB102 y jumpers	1 o más	Solo prototipado; no sustituyen placas soldadas
PC, teclado, pantalla y cables Ethernet	Según montaje	Equipo de desarrollo reutilizado
Cautín, base, pinzas y cortador	Un juego	Herramientas, no componentes de cada Faro
Multímetro	1	Necesario para las mediciones exigidas; modelo y evidencia pendientes

Si se necesitan fuentes nuevas, el presupuesto histórico contemplaba alrededor de CRC 5593 por las dos. El valor de circuitos más esas fuentes sería cercano a CRC 14687. El kit de soldadura y multímetro tenía una referencia histórica de CRC 18306. Son referencias del documento anterior, no precios actualizados.

Justificación de las piezas

- R1 limita la corriente IR. Con 3,3 V y caídas típicas de LED/transistor se espera un orden de 5-6 mA, que se confirma midiendo la caída en R1.
- R2 limita la corriente de base y R3 mantiene apagado Q1 cuando no hay excitación.
- R4 lleva RX a 3,3 V en oscuridad; el PT333 lo lleva hacia GND con luz.
- R5 y R6 limitan corriente de los indicadores; su color no afecta los GPIO.
- C1 desacopla 3,3 V y GND y no tiene polaridad.
- J1 y los cables permiten retirar la Raspberry sin desoldarla.

No se utilizan LM311, LDR, receptor de control remoto de 38 kHz ni alimentación de 5 V en esta versión. La fuente de la Raspberry sí entrega su alimentación normal por micro-USB; eso es distinto de aplicar 5 V a un GPIO.

Referencias de compra históricas

El documento anterior cita CRCibernética y una consulta del 13 de octubre de 2026. Se conservan para localizar categorías y contrastar una compra; los precios y modelos concretos deben verificarse en el comprobante o con el proveedor.

- https://www.crcibernetica.com/infrared-emitters-and-detectors/
- https://www.crcibernetica.com/bjt-transistor-npn-2n2222a/
- https://www.crcibernetica.com/prototype-pcb-5x7cm-one-sided-board/
- https://www.crcibernetica.com/jumper-wire-kit-140pcs/
- https://www.crcibernetica.com/soldering-and-multimeter-kit/

Bitácora de pruebas con multímetro

Integrantes: Jimmy Castillo Guzmán y Juan Carlos Gómez. **Circuito:** esquema GPIO17/27 a 3,3 V, PT333 con pull-up de 10 k. **Instrumento, fecha y responsable:** pendientes de registrar en cada sesión. **Estado:** procedimiento preparado; no se recibieron valores medidos con multímetro.

Las columnas pendientes no equivalen a pruebas aprobadas. Las lecturas `pigs r` son digitales, no mediciones de voltaje ni de continuidad.

Puntos de prueba

Punto	Red
TP1	Alimentación 3,3 V
TP2	GND, referencia de las medidas de tensión
TP3	Base del transistor driver Q1
TP4	Colector de Q1
TP5	RX, colector del PT333 y GPIO27
TP6	Botón, GPIO24

1. Inspección y continuidad, sin alimentación

Desconectar la Raspberry y cualquier fuente. Medir entre pads para comprobar también la soldadura. No esperar un pitido a través de resistencias o diodos.

Comprobación	Criterio	A	B
Soldaduras y alambres	Sin puentes accidentales ni uniones flojas	Pendiente	Pendiente
J1-1 a extremo superior de R4	Continuidad directa de 3V3	Pendiente	Pendiente
J1-2 a emisor Q1 y emisor Q2	Continuidad directa de GND	Pendiente	Pendiente
J1-3 a entrada de R2	Continuidad directa de TX	Pendiente	Pendiente

Comprobación	Criterio	A	B
Salida R2 a base Q1	Continuidad directa	Pendiente	Pendiente
J1-4 a colector Q2 y extremo RX de R4	Continuidad directa de RX	Pendiente	Pendiente
R1 y D1, rama hacia colector Q1	Continuidad en cada segmento sin atravesar el LED o R1	Pendiente	Pendiente
J1-5 y J1-6 a sus resistencias	Continuidad de indicadores	Pendiente	Pendiente
J1-7 a SW1; otro contacto a GND	Continuidad según esquema	Pendiente	Pendiente
Header a pines físicos 1,6,11,13,15,16,18	Cada cable llega a su función	Pendiente	Pendiente

2. Ausencia de cortos, sin alimentación

Medida	Criterio	A	B
VCC-GND	Sin corto estable; C1 puede causar una lectura transitoria	Pendiente	Pendiente
Pads vecinos de J1	Sin puentes ajenos al esquema	Pendiente	Pendiente
RX-GND con detector oscuro	No debe existir puente directo permanente	Pendiente	Pendiente
TX-RX	Sin unión directa	Pendiente	Pendiente
Botón abierto y pulsado	Abierto y continuidad respectivamente	Pendiente	Pendiente

Anotar la resistencia real y la estabilidad de la lectura; no declarar un valor universal VCC-GND porque los semiconductores y el condensador afectan la medida. Si hay corto, corregirlo antes de conectar la Raspberry.

3. Valores de componentes

Para medir una resistencia sin caminos paralelos, levantar una pata si hace falta, siempre sin energía. Confirmar tolerancia con bandas o referencia.

Ref.	Valor esperado	A medido	B medido
R1	330 ohmios	Pendiente	Pendiente
R2	2,2 kiloohmios	Pendiente	Pendiente
R3	10 kiloohmios, base-GND	Pendiente	Pendiente
R4	10 kiloohmios, pull-up RX	Pendiente	Pendiente
R5	1 kiloohmio	Pendiente	Pendiente
R6	1 kiloohmio	Pendiente	Pendiente
D1	Conducción en un sentido, según su ficha	Pendiente	Pendiente
Q1	Patillaje y uniones B-E/B-C de NPN confirmados	Pendiente	Pendiente

Ref.	Valor esperado	A medido	B medido
Q2	Patillaje C/E confirmado	Pendiente	Pendiente

4. Tensiones, con alimentación

Solo después de aprobar continuidad y cortos. Punta negra en TP2, salvo la caída en R1, que se mide entre sus dos extremos. Los valores de la tabla son objetivos de diseño aproximados, no lecturas ya obtenidas ni límites absolutos del fabricante.

Punto y condición	Objetivo	A medido	B medido
TP1	Cercano a 3,3 V	Pendiente	Pendiente
TP3, TX apagado	Cercano a 0 V	Pendiente	Pendiente
TP3, TX encendido	Unión B-E conduciendo, normalmente 0,6-0,8 V	Pendiente	Pendiente
TP4, TX encendido	Bajo si Q1 satura, normalmente inferior a 0,3 V	Pendiente	Pendiente
Caída en R1, TX encendido	Registrar y calcular I=V/330	Pendiente	Pendiente
TP5, detector tapado, otro IR apagado	Alto, próximo a 3,3 V	Pendiente	Pendiente
TP5, IR del otro Faro enfrentado y encendido	Bajo, objetivo inferior a 0,5 V	Pendiente	Pendiente
TP5 en cualquier condición	No superar 3,3 V nominales de la electrónica	Pendiente	Pendiente
TP6, botón suelto y pull-up activo	Cercano a 3,3 V	Pendiente	Pendiente
TP6, botón pulsado	Cercano a 0 V	Pendiente	Pendiente

El IR encendido se fuerza con `pigs modes 17 w` y `pigs w 17 1`; se apaga con `pigs w 17 0`. El receptor permanece como entrada (`pigs modes 27 r`). Detener primero los programas del Faro. Los indicadores se prueban en GPIO22 y GPIO23. Al acabar, apagar salidas de diagnóstico y no dejar programas compitiendo.

5. Cierre de sesión

Fecha	Responsable	Instrumento	Hallazgo	Corrección	Reprueba y evidencia
Pendiente	Pendiente	Pendiente	Pendiente	Pendiente	Pendiente

Registrar fotografías de las puntas y el valor de multímetro para VCC-GND, alimentación y RX con/sin luz. No publicar contraseñas, direcciones de usuarios ni llaves en esas fotos. El índice de evidencias identifica las vistas faltantes.

Bitácora de actividades por estudiante

Método: reconstrucción aproximada a partir de las actividades del proyecto y la estimación del grupo de **unas 20 horas-persona en total**. No se contó con un registro diario completo. La distribución de diez horas por integrante y las fechas por actividad son estimaciones para revisión y aceptación del grupo. No se presentan como horarios comprobados mediante Git.

Se interpretan los tres fines de semana de trabajo como 17-18 y 24-25 de octubre, y 31 de octubre-1 de noviembre de 2026. Algunas implementaciones tienen commits anteriores: estas filas incluyen revisión, ajustes y pruebas de ese código, no afirman que toda función se escribió por primera vez en esa fecha.

Jimmy Castillo Guzmán

Fecha estimada	Actividad	Horas estimadas
2026-10-17	Revisar materiales, resistencias y funciones del emisor y detector	1,5
2026-10-18	Montar prototipo y revisar GPIO, indicadores y botón	1,5
2026-10-24	Preparar distribución y soldar componentes de una placa	2,0
2026-10-25	Revisar uniones, jumpers y alineación óptica del prototipo	1,0
2026-10-31	Completar montaje soldado y adaptar el puerto a GPIO17/27	2,0
2026-11-01	Configurar Raspberry, probar por SSH y revisar evidencias y documentación	2,0
TOTAL	Jimmy	**10,0**

Juan Carlos Gómez

Fecha estimada	Actividad	Horas estimadas
2026-10-17	Revisar tramas, CRC y pruebas de la biblioteca del Faro	2,0
2026-10-18	Revisar directorio firmado, rutas y simulaciones de la mesh	2,0
2026-10-24	Ajustar y probar cifrado por capas y comandos del chat	2,0
2026-10-25	Revisar código y configuración del Worker, gateway y pruebas locales	2,0
2026-10-31	Apoyar revisión de integración, materiales y casos de prueba	1,0
2026-11-01	Revisar RFC, resultados y documentación de entrega	1,0
TOTAL	Juan Carlos	**10,0**

Resumen

Integrante	Horas-persona estimadas
Jimmy Castillo Guzmán	10,0
Juan Carlos Gómez	10,0
TOTAL DEL GRUPO	**20,0**

Estas veinte horas son la suma de tiempo por persona, no veinte horas de trabajo simultáneo de ambos. La cifra total fue aportada por el grupo; el reparto es una reconstrucción. Antes de entregar, corregir cualquier actividad o fecha que no represente el trabajo de cada integrante.

Registro previo conservado, fuera del total

La bitácora original contenía estas dos filas sin identificar estudiante. Pueden estar incluidas en la estimación global; no se suman otra vez.

Fecha original	Actividad original	Horas	Atribución
2026-10-12	Instalar Python, Node, Git y VS Code	0,5	Sin especificar
2026-10-12	Crear estructura del repositorio y .gitignore	0,5	Sin especificar
SUBTOTAL HISTÓRICO	No adicional a las 20 h	1,0	Sin asignar

Bitácora de pruebas y resultados

Fecha de corte: 1 de noviembre de 2026. Se separan ejecución observada, información reportada por el grupo y procedimientos pendientes. No se guardan direcciones IP, direcciones físicas ni identidades reales de usuarios del sistema.

1. Pruebas automatizadas ejecutadas

Prueba	Comando	Resultado observado	Qué no demuestra
Python	`python -m pytest tests -q`	**140 aprobadas**, 12,81 s en la ejecución de revisión	Luz, soldadura o voltajes reales
Worker local	`npx vitest run`	**23 aprobadas**, una suite	Uso del despliegue público o integración real
TypeScript	`npx tsc --noEmit`	Código de salida 0, sin errores	Publicación ni disponibilidad de la URL

Entorno de revisión: PC Windows, Python 3.12, dependencias del proyecto y runtime local de Cloudflare para Vitest. Se añadieron cinco pruebas del puerto pigpio a las 135 de Python que tenía la base. Los GPIO se simulan en esas cinco pruebas.

Una primera ejecución restringida no pudo usar correctamente sockets y carpetas temporales. Se repitió con acceso al entorno local y pasaron las 140; no fue necesario alterar aserciones ni marcar pruebas como omitidas para obtenerlo.

Las pruebas cubren CRC y tamaños, tramas externas, ACK, duplicados, directorio, rutas, cebolla, chat, pánico y gateway. Las del Worker incluyen firmas, replay, WebSocket, Atom, TTL, borrado al leer, validación de entradas y ausencia de columnas de registro de IP/User-Agent. Las firmas de prueba son fixtures, no llaves de producción.

2. Desarrollo físico reportado

Etapa	Resultado reportado	Clasificación
Prototipo	Indicadores, botón y enlace IR local probados	Reporte y capturas de desarrollo
Velocidad local	Tramas estables a 1000 baudios y fallos al superar aproximadamente 1050	Reporte del grupo, sin tabla formal de BER
Haz tapado	Se reportó una prueba local correcta usando material opaco	Desarrollo; no reemplaza pruebas entre placas
Placa perforada	Se recibió foto de componentes de una placa soldada	Evidencia visual parcial
Paso a dos Faros	Intentos sin confirmación de recepción válida	Integración pendiente

La velocidad UART es una configuración, no la tasa útil. El resultado local no se extrapola a otra distancia, otra placa o luz ambiental distinta.

3. Diagnóstico observado por SSH

Se controlaron ambas Raspberry a través de SSH, sin modificar el código del proyecto instalado. Se alternaron los emisores por GPIO17 y se leyó GPIO27.

Comprobación	Resultado observado
TX en A y B	pigpio generó una secuencia UART de 14 bytes a 300 baudios en unos 0,48 s
Recepción B sin pull interno	Datos repetitivos e inestables que no siguieron claramente al emisor
Recepción A	Sin datos ni transiciones durante la secuencia de B
Repetición con pull-up interno	Cero bytes y cero transiciones útiles en ambos sentidos
Alineación repetida A hacia B	B permaneció en 1 al apagar y encender A con pull-up; sin él fluctuó

Estos resultados no verifican que el LED emita correctamente solo porque exista una onda GPIO. Tampoco identifican de forma concluyente una conexión rota. Se requiere revisar el camino óptico y medir RX, pull-up, alimentación y GND. El estado del enlace físico no se sustituye por la suposición «todo funciona».

4. Protocolo de medición formal, pendiente

El enunciado exige tasa efectiva y BER a tres distancias bajo luz ambiental normal. No obliga a que una de ellas sea un metro salvo para el extra. Se proponen **1 cm, 2 cm y 5 cm**, manteniendo alimentación y resistencias iguales. Cambiar esas distancias si el montaje requiere otras, y registrar las usadas.

Usar 100 tramas para BER y 20 para tasa con ACK por distancia y sentido. Registrar el tiempo real, baudios, tipo de iluminación, hora y cualquier pérdida. Los comandos completos están en el manual. No se llena la tabla con valores teóricos.

Distancia	Sentido	Tramas recibidas/enviadas	Bits distintos/totales	BER	Tasa útil B/s
1 cm	A hacia B	Pendiente	Pendiente	Pendiente	Pendiente
1 cm	B hacia A	Pendiente	Pendiente	Pendiente	Pendiente
2 cm	A hacia B	Pendiente	Pendiente	Pendiente	Pendiente
2 cm	B hacia A	Pendiente	Pendiente	Pendiente	Pendiente
5 cm	A hacia B	Pendiente	Pendiente	Pendiente	Pendiente
5 cm	B hacia A	Pendiente	Pendiente	Pendiente	Pendiente

Condiciones por sesión: fecha y hora, luz ambiental, instrumento de distancia, baudios, R4 final, número de tramas, alineación y responsable: **pendientes**.

BER = bits distintos / bits totales comparados. El programa reconstruye el contenido esperado por semilla y SEQ y contabiliza las tramas perdidas como todos sus bits erróneos. Es una estimación conservadora basada en tramas, no una medida de cada bit físico 8N1 ni una separación estadística de pérdida y error de bit. La tasa útil es payload confirmado dividido por segundos de la prueba, incluyendo confirmaciones y reintentos.

Una trama máxima produce 267 bytes UART incluyendo relleno pigpio. A 300 baudios su tiempo de transmisión teórico es 267 x 10 / 300 = **8,9 s**; no es una medición de rendimiento. Una tasa teórica tampoco se copia en la columna de tasa útil.

5. Casos de aceptación de la integración

Caso	Resultado esperado	Estado físico/integrado
Texto A-B y B-A	Texto completo visible en receptor	Pendiente
ACK dirigido	Emisor informa entrega confirmada	Pendiente
Dos comunidades solo por Faro	Mensaje cruza luz; al bloquearla falla el camino	Pendiente
Tres relays	Ruta con candidatos suficientes y sin modo directo	Probado en software; pendiente por luz
Privado/tema	Solo destinatario o miembros descifran	Pruebas software; pendiente defensa
Pánico físico	Borrado lógico, renovación y señal visual	Pruebas software; pendiente botón sobre sistema integrado
Mensaje público al Worker	Sala y feed reciben solo público	**No utilizado por el grupo**
Buzón muerto	Blob fijo, primera lectura y borrado	Probado localmente en suite; uso integrado no realizado

6. Incidencias y tratamiento

Incidencia	Tratamiento aplicado o pendiente
Archivo no encontrado	Ejecutar módulos desde la raíz de su repositorio

Incidencia	Tratamiento aplicado o pendiente
GPIO ya en uso	Detener otro lector y liberar puerto antes de abrirlo
Polaridad distinta del receptor original	Invertir RX en el puerto pigpio
Scripts de prueba con GPIO14/15	Retirar `tools/emisor.py` y `tools/receptor.py`
Mensaje «enviado» sin texto al otro lado	Comprobar recepción; difusión no confirma
Lectura depende del pull interno	Medir acondicionamiento y revisar conexión, sin inferir una causa única
Varias tramas sin separar	Usar pausa de un segundo a 300 baudios en BER
Bitácora incompleta al final	Reconstrucción señalada como estimada; registrar a futuro al trabajar

Autoevaluación y limitaciones

Estado de entrega

Se prepararon fuentes, manual, RFC, esquema, BOM, bitácoras y PDFs. Las pruebas locales aprueban 140 casos de Python y 23 del Worker y TypeScript no reporta errores. No se considera validado el enlace entre placas ni la publicación desde la mesh al Worker. El grupo indicó que el Worker se hizo, pero no se utilizó.

Faltan medidas reales de multímetro, resultados de tasa/BER a tres distancias y fotografías completas de ambas caras de ambos Faros. Estas evidencias no se reemplazan por resultados esperados. Las horas son una reconstrucción aproximada de la estimación de veinte horas-persona proporcionada por el grupo.

Rúbrica del proyecto, escala 0 a 10

Las notas siguientes son una **propuesta de autoevaluación documental** basada en la evidencia disponible. Ambos estudiantes deben revisarlas antes de entregar. No son calificaciones del profesor ni presuponen que todas las pruebas físicas pasaron. El peso corresponde a la rúbrica interna de la página 8 del enunciado.

Rubro	Peso	Propuesta 0-10	Justificación
Hardware soldado, esquema, BOM y pruebas	25%	4	Esquema y materiales disponibles; foto parcial y enlace sin validar
Capa 1 y DispositivoLuzAdaptador	10%	6	Códec y adaptador probados; recepción física pendiente
Capas 2 y 3, mesh y directorio	15%	8	Pruebas de rutas, firma y comunidades simuladas; falta defensa por luz
Privacidad	10%	7	Cebolla, padding y pánico; límites documentados y botón integrado pendiente
Aplicación anónima	5%	8	Chat y comandos probados por software; no validado todo el recorrido físico
Aplicación pública en edge	25%	6	Implementada y probada localmente; no utilizada en la integración
RFC	5%	9	ASCII, protocolos, campos, versiones, privacidad y seguridad

Rubro	Peso	Propuesta 0-10	Justificación
Documentación	5%	8	Apartados y anexos completos; evidencias físicas aún pendientes
TOTAL PONDERADO PROPUESTO	100%	**62,5/100**	Suma de nota/10 por peso

La página 2 menciona 30% de valor de la asignación y la página 8 dice «Proyecto 20%». No se resuelve esa diferencia inventando un peso: debe confirmarse con el profesor. La tabla anterior solo normaliza la rúbrica interna a cien puntos. No se asignan extras de kick-off ni velocidad. Una reconstrucción de kick-off no prueba que se entregó cuando correspondía.

Aprendizaje, escala 1 a 5

Escala del enunciado: 1 muy malo, 2 malo, 3 regular, 4 bueno y 5 muy bueno. Estas son propuestas para reflexión; cada integrante debe aceptar o corregir su propia valoración. No son resultados de una encuesta realizada.

Rubro	Jimmy, propuesta	Juan Carlos, propuesta
Soldadura y hardware	4	3
Capa física	4	4
Enrutamiento mesh	3	4
Privacidad y anonimato	4	4
Serverless en el edge	3	4
Redacción de RFC	4	4

Problemas y limitaciones

- La conmutación manual del detector no asegura tramas a una velocidad determinada.
- La iluminación, alineación, reflejos y conexiones del pull-up afectan el margen de RX.
- Una entrada que cambia solo con la resistencia interna puede estar flotante; hay que medir antes de concluir.
- El lector puede quedar ocupado tras una interrupción; se requiere un solo dueño del GPIO.
- El silencio de veinte bytes obliga a separar ráfagas de BER a velocidades bajas.
- La red requiere siete nodos para la política de cinco candidatos y tres relays.
- A 300 baudios la fragmentación y los anuncios de control son lentos; su integración física no está validada.
- El Worker no se utilizó en la demostración, aunque tiene código, configuración y URL informada.
- HELLO/RUTA no están firmados; existe exposición a Sybil y manipulación de rutas.
- El directorio revela direcciones de transporte y la cebolla no elimina correlación global de tiempos.
- El pánico no garantiza borrado forense en microSD ni de todas las copias de memoria.
- El origen de un privado no se autentica solo con la pista que muestra el chat.

El reporte de commits acompaña esta autoevaluación y conserva los autores del historial. No se usa el número de commits para determinar la nota ni las horas.

Reporte de commits de Git

Corte: historial previo al commit de esta entrega, incluyendo la rama que contenía las pruebas UART retiradas. Los títulos y autores se conservan como aparecen en Git. El hash del commit documental se consulta en GitHub; no se intenta incluir un commit dentro de su propio contenido.

Resumen por autor del historial

Autor registrado	Commits
Jimmy Castillo Guzman	1
Jimmy Castillo Guzmán	3
JimmyCR1	1
JuankGO	14

Los nombres JimmyCR1 y las variantes de Jimmy Castillo corresponden a firmas distintas del historial; no se convierten automáticamente en horas. Se incluyen merges y commits de código, por lo que los conteos no miden esfuerzo.

Detalle

Commit	Fecha	Autor	Descripción
891e5eb	2026-10-30	Jimmy Castillo Guzmán	test(faro): scripts de prueba cruda de emisor y receptor por UART
c7a3b9b	2026-10-30	JuankGO	Merge pull request #4 from JuankGO/faro-simplificado
f4538c1	2026-10-30	Jimmy Castillo Guzmán	feat(faro): modo loopback en faro.medir para probar un solo Faro
77cf357	2026-10-30	JuankGO	Merge pull request #3 from JuankGO/main
8a30954	2026-10-30	JuankGO	Merge pull request #2 from JuankGO/faro-simplificado
ba70abd	2026-10-30	Jimmy Castillo Guzmán	docs(faro): documentar el Faro simplificado sin comparador
1c7e8a9	2026-10-26	Jimmy Castillo Guzman	Merge pull request #1 from JuankGO/ACACHETE
f054099	2026-10-26	JimmyCR1	build(Nuevo): Juank no sabe nada, yo si
f9e5990	2026-10-17	JuankGO	Worker de Cloudflare desplegado con D1 y Durable Objects
ffdf247	2026-10-16	JuankGO	Integración Mesh + Chat completada
ce04044	2026-10-15	JuankGO	Prueba de integracion de la mesh
8ecfa5c	2026-10-14	JuankGO	Directorio firmado con TTL y tabla de vecinos
972d72a	2026-10-12	JuankGO	Parte 7 completa: identidad, panico y cebolla

Commit	Fecha	Autor	Descripción
55c0558	2026-10-12	JuankGO	Parte 7A: identidad criptografica del nodo (Ed25519 + Curve25519)
386170c	2026-10-12	JuankGO	Modelo de amenazas v1
064cd70	2026-10-12	JuankGO	Demo del medio simulado entre dos Faros
fe44373	2026-10-12	JuankGO	Capa 1: trama Faro v1 con CRC, Manchester y DispositivoLuzAdaptador
10139f8	2026-10-12	JuankGO	Estructura inicial del repositorio
b77a1c2	2026-09-30	JuankGO	Initial commit

Reproducir el reporte

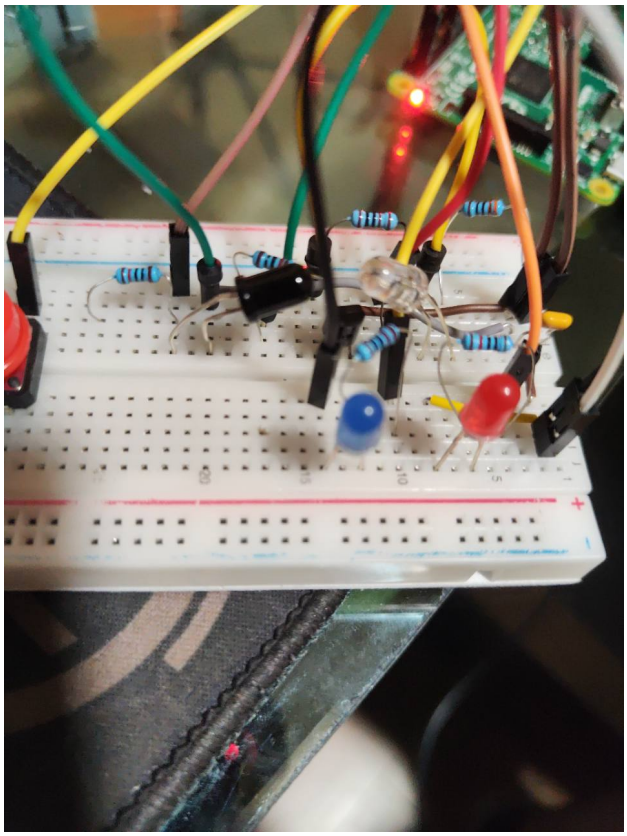
```
git log --all --date=short --format="%h %ad %an %s"
git shortlog -s -n --all
git log --all -- tools/emisor.py tools/receptor.py
```

La última orden permite revisar el alta y la retirada de los dos scripts. Su eliminación no reescribe el historial. Las pruebas oficiales se conservan como módulos del paquete faro.

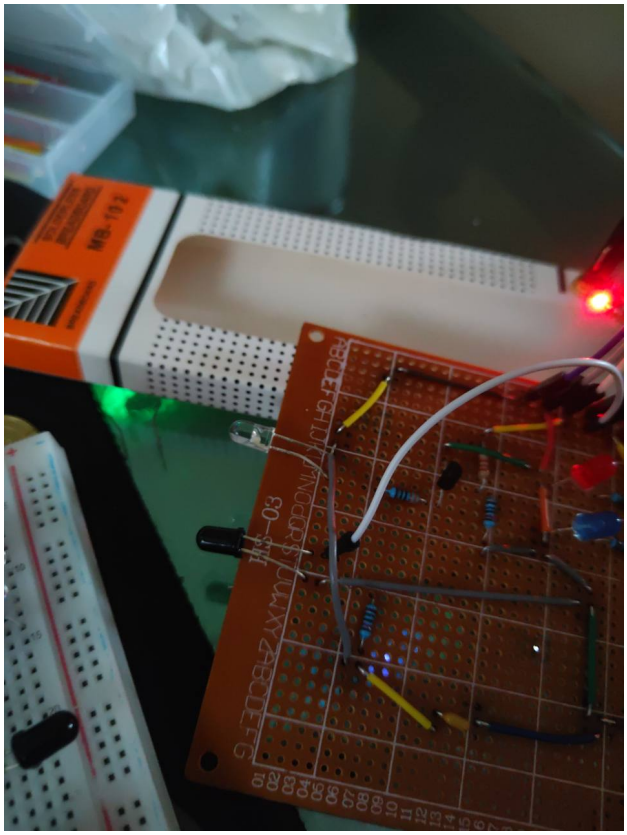
Inventario de fotografías

Se conservan fotografías proporcionadas por el grupo sin retocarlas para simular resultados. Las vistas de protoboard se identifican como desarrollo.

Archivo o vista requerida	Contenido y estado
`prototipo-a.jpg`	Prototipo en MB102; evidencia de desarrollo, no placa final
`faro-b-componentes.jpg`	Placa soldada, lado de componentes; no verifica el cobre
Faro A, cara de componentes final	Pendiente recibir
Faro A, cara de cobre final	Pendiente recibir
Faro B, cara de cobre final	Pendiente recibir
Continuidad, VCC-GND y voltajes con multímetro	Pendiente recibir



Prototipo A en protoboard



Placa soldada B, lado de componentes

Para completar la entrega, fotografiar ambas caras de cada placa sin alimentación, con etiquetas A/B y suficiente enfoque para distinguir soldaduras y puentes. Las fotos de medición deben mostrar el punto y la lectura. Evitar capturar contraseñas, llaves privadas o identificadores de usuarios.

Lista de entrega y trazabilidad del enunciado

Fuente: «Proyecto 1 - FaroNet», septiembre de 2026. La columna de evidencia identifica implementación y documentos; no convierte un requisito en aprobado. «Pendiente físico» significa que falta demostrarlo o adjuntar el registro.

Requisitos funcionales

Requisito y página	Evidencia o ubicación	Estado al corte
Emisor, receptor y acondicionamiento, p.3	Esquema y BOM, GPIO17/27 y PT333	Documentados; validación física pendiente
Indicadores y botón físico, p.3	`faro/gpio.py`, TX22/RX23 y botón24	Código presente; prueba integrada pendiente
Placa soldada y headers desmontables, p.3	Foto de componentes y esquema J1	Evidencia parcial; no usar protoboard en revisión final
Esquemático y BOM en colones, p.3	SVG, conexiones y BOM	Preparados; precios orientativos
Fotos de ambas caras, p.3	Índice de fotografías	Faltan vistas finales de A y cobre de B
Continuidad, cortos y voltajes, p.3	Bitácora de multímetro	Procedimiento listo, mediciones pendientes
Trama máxima 128 B, CRC y versión, p.4	`faro/frame.py`, RFC 2	Pruebas software
Biblioteca DispositivoLuzAdaptador, p.4	`faro/dispositivo_luz_adaptador.py`	Implementada y probada por software
Acceso al medio, p.4	Espera de silencio y backoff	Implementado; no elimina toda colisión
Dirección física persistente y rotable, p.4	`faro/direccion.py`, `mesh/nodo.py`	Implementada y probada por software
BER y tasa a tres distancias, p.4	Bitácora de pruebas y RFC, apéndice B	Pendiente medir
Ethernet/WiFi y Faro, p.4	UDP y `mesh/enlace_faro.py`	Software probado; puente óptico pendiente
Comunidades unidas exclusivamente por Faro, p.4	Test de dos comunidades y topología del manual	Simulado; falta prueba física sin ruta alternativa
Enrutamiento dinámico, p.4	`mesh/rutas.py`, `mesh/nodo.py`	Pruebas software
Directorio con entradas firmadas, p.4	`mesh/directorio.py`	Pruebas de firma, estructura y TTL
Modelo de amenazas, p.4	`docs/amenazas.md`, informe y RFC 7	Documentado
Anonimato del origen y tres saltos, pp.4-5	`mesh/onion.py`, `saltos.py`	Software probado; límites explícitos
Cifrado con biblioteca reconocida, p.5	PyNaCl y WebCrypto	Implementado; sin primitivas propias
Padding a tamaño fijo, p.5	Celda1024 y mensaje256	Pruebas software
Botón de pánico, p.5	Borrado lógico, renovación y parpadeo	Pruebas software; validar hardware

Requisito y página	Evidencia o ubicación	Estado al corte
Sin bitácoras de usuarios y TTL, p.5	Tablas acotadas, limpieza y Worker	Implementado; directorio operativo expone direcciones
Chat privado y temas privados internos, p.5	`app/` y filtro del gateway	Pruebas software
Relay autenticado HTTPS/WSS, p.5	`gateway/`, Ed25519 y nonce	Implementado; no utilizado integrado
Salas DO/WebSockets y feed Atom, p.5	`worker/src/index.ts`	Pruebas locales aprobadas
Buzón opaco cifrado, fijo, TTL y borrado, p.5	Endpoint y `tools/prueba_buzon.py`	Pruebas locales aprobadas; cifra el cliente
Directorio público verificable, p.5	`/directorio` y firmas	Pruebas locales
Rate limiting y no registro, p.5	Binding LIMITE, D1 y observabilidad apagada	Código y pruebas; no se afirma invisibilidad al proveedor
Despliegue reproducible sin secretos, p.6	Wrangler, migraciones y manual	Configuración presente; Worker no utilizado
RFC ASCII de Faro y FaroNet, p.6	`docs/rfc/rfc-faronet.txt`	Fuente y PDF
Trama externa y longitudes declaradas, p.6	`enviar_crudo`, LEN y cabeceras	Pruebas software
Parámetros por terminal, p.6	argparse de nodos, chat y herramientas	Implementado

Documentación y administración

Entregable, pp.7-8	Ubicación	Estado
Introducción, ambiente y diseño	Informe, secciones 1-3	Preparado
Funciones y estructuras	Informe y API	Preparado
Comandos concretos	Manual	Preparado
Actividades, fechas, horas y totales por estudiante	Bitácora de horas	Reconstrucción estimada de 20 h-persona
Estado, limitaciones y commits	Autoevaluación y reporte de Git	Preparado
Rúbrica 0-10 y aprendizaje 1-5	Autoevaluación	Propuesta para revisión del grupo
Lecciones y bibliografía	Informe, secciones 8-9	Preparado
Código documentado	Docstrings y API	Presente
Markdown/ASCII y PDFs	`docs/` y `output/pdf/`	Generados desde fuentes
URL pública	Manual e informe	Informada, no usada en demostración
Kick-off	`KICKOFF.md`	Reconstruido; no acredita entrega histórica

Cierre antes de enviar al profesor

- Completar los valores reales de multímetro, no los objetivos de diseño.

- Confirmar ambos sentidos y medir tasa útil y BER a tres distancias reales.
- Adjuntar las cuatro vistas finales de las placas soldadas.
- Revisar y aceptar la distribución estimada de horas y las notas propuestas.
- Mantener explícito que el Worker no se utilizó en la integración.
- Volver a generar los PDFs después de completar evidencias.
- Comprobar que no se suben contraseñas, llaves, `.dev.vars` ni registros de usuarios.
- Llevar el Faro físico, funcionando y soldado; la defensa dura como máximo veinte minutos.

El enunciado asigna cero a hardware si no se presenta soldado, cero si no hay documentación y cero si el código no compila. El grupo debe revisar el formato de envío y las fechas de la plataforma; no están fijadas en este PDF.

Kick-off: reconstrucción del planteamiento inicial

Nota de alcance: este documento se reconstruye al preparar la entrega. No demuestra que se presentó en la clase o fecha exigida. No se solicita por ello el extra ni se oculta la falta de evidencia de entrega histórica.

1. Introducción y estrategia

Construir dos interfaces de comunicación por luz y usarlas para transportar una red con privacidad. Dividir el trabajo en hardware, códec, mesh, cifrado, aplicación y edge permite detectar errores antes de integrar todo.

```
Faro soldado -> tramas con CRC -> mesh -> cebolla y chat
                                       -> gateway -> Worker publico
```

Los privados y temas privados quedan en FaroNet. Solo contenido marcado como público puede salir al edge. El hardware se prototipa y luego se suelda con conectores desmontables.

2. Ambiente y flujo de trabajo

Raspberry Pi 2 y Raspberry Pi OS; Python para nodos y GPIO; TypeScript con Wrangler para Cloudflare. PC con Git, editor y terminal. Para el circuito: protoboard, cautín, estaño, pinzas, placa perforada y multímetro. El debugging avanza de voltajes y GPIO a tramas y, después, a pruebas de red.

El flujo es implementar una parte, ejecutar sus pruebas, revisar el cambio y publicar un commit. Las llaves y contraseñas quedan fuera de Git. Los commits identifican contribuciones, no sustituyen una bitácora de horas.

3. Control de versiones

Repositorio del grupo: https://github.com/JuankGO/redes-proyecto1-faronet

La rama principal concentra la versión de entrega. Se conservaron las contribuciones del historial. Los scripts crudos de UART se retiran para usar una sola biblioteca y el mapa GPIO17/27 del montaje actual.

4. Diseño inicial y amenazas

El esquema actual está en el anexo de hardware: LED IR y NPN para transmitir, PT333 con pull-up para recibir, dos indicadores y botón. El componente central es una Raspberry que ejecuta la aplicación y el control de la placa por GPIO.

Adversarios considerados desde el diseño: un nodo malicioso, un observador óptico, el operador del gateway y el proveedor del edge. Se proponen cifrado de extremo a extremo con bibliotecas reconocidas, tres relays, tamaños fijos, dirección rotable, pánico y TTL. Persisten riesgos de correlación temporal, colusión, sistema operativo comprometido y recuperación de datos en microSD.

Resultado posterior

El código de cada capa y las pruebas locales existen. El Worker se implementó pero el grupo informó que no lo utilizó en la demostración integrada. Las mediciones de hardware y la comunicación entre placas no se declaran aprobadas sin evidencia. El informe final desarrolla estas decisiones y limitaciones.